

Dalil

Guide d'utilisation du projet

Version du document : 1.0

Version du projet : 0.1.0 (squelette)

Date : 20 octobre 2026

Dépôt : github.com/hamzasinko/dalil

Licence : MIT

Sommaire

1. Présentation du projet
   Objectif, périmètre et état actuel
2. Structure des fichiers
   Rôle de chaque dossier et fichier de configuration
3. Installation et environnement
   Python, venv, extras et pré-commit
4. Commandes à connaître
   Tests, lint, typage, build et packaging
5. Workflow Git
   Branches, commits conventionnels et pull requests
6. Workflow GitHub
   Protections, CI/CD, releases, sécurité et Dependabot
7. Publier une version
   PyPI Trusted Publishing et création de release
8. Roadmap suggérée
   Phases, branches et critères d'acceptation
9. Checklist et dépannage
   Avant chaque PR et erreurs fréquentes

1. Présentation du projet

Dalil (« guide » en arabe) est une bibliothèque Python open source qui rend l'exécution du code compréhensible. Un seul décorateur, `@dalil.trace`, transforme une fonction en un ensemble d'outils pédagogiques :

1. un **flowchart** construit à partir de l'AST ;
2. une **trace d'exécution** ligne par ligne (variables, branches, retours) ;
3. une **estimation de complexité** temporelle et spatiale ;
4. un **rapport HTML** autonome ;
5. un **export LaTeX** au format `algorithm2e` / `algorithmicx`.

État actuel

La version 0.1.0 est un **squelette** : l'API publique et les modèles de données existent, les couches métier lèvent encore `NotImplementedError`. Ces comportements sont couverts par des tests `xfail(strict=True)`, ce qui garantit qu'ils seront implémentés sans oublier les tests.

Élément	Valeur
Version	0.1.0 (dynamique, lue dans `src/dalil/__init__.py`)
Python	3.9 à 3.13
Backend de build	hatchling
Dépendance runtime	`jinja2` uniquement
Tests	96 réussis, 19 xfailed, couverture 100 %
Branche par défaut	`dev`

Public visé : enseignants, étudiants, auteurs de publications et toute personne qui veut expliquer un algorithme à partir de code réel plutôt que d'un pseudo-code.

2. Structure des fichiers

```
Dalil/
├── src/dalil/                  # code de la bibliothèque
│   ├── __init__.py             # API publique + __version__
│   ├── decorator.py            # @trace, classe TracedFunction
│   ├── tracer/                 # événements et historique d'exécution
│   ├── flowchart/              # AST -> graphe -> DOT / Mermaid / SVG
│   ├── complexity/             # estimation statique temps / espace
│   ├── report/                 # génération HTML (Jinja2)
│   │   └── templates/report.html.j2
│   ├── latex/                  # AST -> algorithme LaTeX
│   ├── exceptions.py           # hiérarchie DalilError
│   ├── _types.py               # alias de types partagés
│   └── py.typed                # marqueur PEP 561
├── tests/
│   ├── unit/<module>/          # tests nominaux
│   ├── edge/<module>/          # cas limites
│   ├── conftest.py             # fixtures partagées
│   └── samples.py              # fonctions d'exemple
├── docs/                       # pages de documentation (index, architecture, usage)
├── examples/basic_usage.py     # exemple exécutable
├── .github/
│   ├── workflows/ci.yml        # lint + matrice de tests + gate + build
│   ├── workflows/release.yml   # test + build + publication PyPI + release
│   ├── ISSUE_TEMPLATE/         # bug_report, feature_request, config
│   ├── PULL_REQUEST_TEMPLATE.md
│   ├── CODEOWNERS
│   └── dependabot.yml
├── pyproject.toml              # métadonnées, extras, ruff, mypy, pytest
├── .pre-commit-config.yaml     # hooks locaux
├── README.md                   # vitrine du projet
├── CONTRIBUTING.md             # guide contributeur
├── CODE_OF_CONDUCT.md          # Contributor Covenant 2.1
├── SECURITY.md                 # politique de sécurité
├── CHANGELOG.md                # Keep a Changelog
├── LICENSE                     # MIT
├── .gitignore / .gitattributes
└── AGENTS.md                   # contexte assistants (local, ignoré par Git)
```

Rôle des dossiers clés

Chemin	Rôle	À modifier quand…
`src/dalil/`	Code livré dans le wheel	Vous ajoutez une fonctionnalité
`tests/unit/`	Comportement nominal	Vous ajoutez une API publique
`tests/edge/`	Cas limites (vide, récursion, générateurs, threads…)	Toute nouvelle fonctionnalité
`docs/`	Documentation MkDocs	Une API ou un concept change

`examples/`	Démonstrations exécutables	Une fonctionnalité devient utilisable
`.github/`	CI, releases, templates, Dependabot	Le processus de contribution change
`pyproject.toml`	Source unique de configuration	Dépendances, outils, métadonnées

AGENTS.md n'est pas versionné : il est listé dans `.gitignore` et sert uniquement de contexte local pour les assistants IA. Il ne doit pas être poussé sur les branches distantes.

3. Installation et environnement

Prérequis

- Python 3.9 ou plus récent (3.13 recommandé pour le développement) ;
- Git et, si possible, GitHub CLI (`gh`) ;
- aucun service externe n'est requis pour les tests.

Mise en place

```
git clone https://github.com/hamzasinko/dalil.git
cd dalil

python -m venv .venv
# Windows PowerShell
.\.venv\Scripts\Activate.ps1
# Linux / macOS
source .venv/bin/activate

python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
pre-commit install
```

Extras disponibles

Extra	Contenu	Usage
`dev`	pytest, pytest-cov, hypothesis, ruff, mypy, pre-commit, build, twine	Développement et CI
`viz`	graphviz	Rendu SVG des flowcharts (à venir)
`cli`	rich	Sortie console enrichie (à venir)
`docs`	mkdocs-material	Site de documentation (mkdocs.yml à créer)

Sous Windows, si PowerShell bloque l'activation du venv, exécutez une fois : `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` .

L'installation est éditable : toute modification dans `src/dalil/` est immédiatement visible, sans réinstallation.

4. Commandes à connaître

Commande	Effet	Quand
`python -m pytest`	Tests + couverture (échec sous 90 %)	Avant chaque commit
`python -m pytest tests/unit`	Tests nominaux uniquement	Pendant le développement
`python -m pytest tests/edge`	Cas limites uniquement	Vérifier la robustesse
`python -m pytest -k nom_du_test -vv`	Un test ciblé	Débogage
`python -m ruff check .`	Lint (E, F, I, B, UP, N, D)	Avant chaque commit
`python -m ruff format .`	Formatage (88 colonnes)	Avant chaque commit
`python -m ruff format --check .`	Vérifie le formatage	Avant une PR
`python -m mypy --strict src tests`	Typage strict	Avant une PR
`python -m mypy src`	Typage du code livré	Vérification rapide
`python -m build`	Construit sdist + wheel dans `dist/`	Avant une release
`python -m twine check dist/*`	Valide les métadonnées PyPI	Après `build`
`pre-commit run --all-files`	Exécute tous les hooks	Avant de pousser
`python examples/basic_usage.py`	Exemple d'utilisation	Vérification manuelle

Séquence complète de vérification

```
python -m pip install -e ".[dev]"
python -m ruff check .
python -m ruff format --check .
python -m mypy src
python -m pytest
python -m build
python -m twine check dist/*
```

Ce que la CI exige réellement : `ruff check`, `ruff format --check`, `mypy --strict src tests`, `pytest` sur Python 3.9 à 3.13 sous Linux, Windows et macOS, puis `build` + `twine check`. Les jobs requis sont `lint`, `test` et `build`.

Ce que produit le build

- `dist/dalil-0.1.0.tar.gz` : source distribution ;
- `dist/dalil-0.1.0-py3-none-any.whl` : wheel contenant le code, `py.typed`, les templates Jinja2 et la licence MIT.

5. Workflow Git

Branches

Branche	Rôle	Règles
`main`	Code stable et revu	PR uniquement, CI verte
`dev`	Intégration (branche par défaut)	PR uniquement, point de départ des features
`prod`	Releases	Un push déclenche test + build + publication PyPI
`feature/<nom>`	Nouvelle fonctionnalité	Part de `dev`, PR vers `dev`
`fix/<nom>`	Correction de bug	Part de `dev`, PR vers `dev`
`docs/<nom>`	Documentation	Part de `dev`, PR vers `dev`

Cycle de travail type

```
# 1. Se placer sur dev et récupérer les dernières modifications
git switch dev
git pull --ff-only

# 2. Créer la branche de travail
git switch -c feature/tracer-core

# 3. Coder + tester
python -m pytest
python -m ruff check .
python -m ruff format --check .
python -m mypy --strict src tests

# 4. Commiter (Conventional Commits)
git add -A
git commit -m "feat(tracer): record locals on each line event"

# 5. Pousser et ouvrir la PR vers dev
git push -u origin feature/tracer-core
gh pr create --base dev --fill

# 6. Après validation et CI verte : squash merge
gh pr merge --squash --delete-branch
```

Commits conventionnels

Préfixe	Usage	Exemple
`feat`	Nouvelle fonctionnalité	`feat(flowchart): link decision branches`
`fix`	Correction	`fix(tracer): restore sys.settrace on error`

`test`	Tests	`test: cover stacked decorators`
`docs`	Documentation	`docs: explain the template context`
`refactor`	Refactorisation sans changement de comportement	`refactor(decorator): extract binding helper`
`chore` / `ci`	Maintenance, outillage	`ci: cache pip dependencies`

Règle d'or : le comportement et la valeur de retour d'une fonction décorée ne doivent jamais changer. Toute fonctionnalité doit être accompagnée de tests nominaux *et* de cas limites.

6. Workflow GitHub

Protection des branches

Les branches `main`, `dev` et `prod` exigent :

- une pull request (pas de push direct) ;
- 1 revue approuvée (les reviews obsolètes sont réinitialisées) ;
- les checks requis `lint`, `test` et `build` ;
- un historique linéaire (squash ou rebase uniquement) ;
- aucun force-push ni suppression de branche ;
- la résolution des conversations avant merge.

Compte solo : GitHub interdit d'approuver sa propre PR. Comme `enforce_admins` est désactivé, le mainteneur peut merger via le bypass administrateur (`gh pr merge --squash --admin`). Dès qu'un second mainteneur est ajouté, activez `enforce_admins` et exigez une vraie revue.

Sur un dépôt personnel, GitHub refuse les restrictions de merge par utilisateur/équipe sur `prod` (erreur 422). Seuls les collaborateurs avec accès en écriture peuvent merger. Pour une restriction explicite, transférez le dépôt vers une organisation.

CI : jobs et déclencheurs

Job	Contenu	Résultat attendu
`lint`	ruff check, ruff format --check, mypy strict	Rapide, premier retour
`tests`	Matrice Python 3.9-3.13 x Linux/Windows/macOS (15 jobs)	Tests + coverage.xml en artefact
`test`	Agrégation de la matrice	Check unique et stable pour la protection
`build`	python -m build + twine check	Artefact `dist`

Déclencheurs : push sur `main`, `dev` et `prod` ; pull requests ciblant `main`, `dev`, `prod` ou `feature/**`. Une nouvelle exécution annule la précédente sur la même référence.

Fonctionnalités GitHub activées

Fonction	État	Usage
Issues + templates	Actives	Bug report et feature request guidés
Discussions	Actives	Questions d'usage et idées
Dependabot (pip + Actions)	Hebdomadaire	PR automatiques de mise à jour
Alertes Dependabot + security updates	Activées	Correctifs de dépendances
Secret scanning + push protection	Activés	Blocage des secrets

Security advisories	Disponibles	Signalement privé des vulnérabilités

Rôle des fichiers GitHub

- `.github/workflows/ci.yml` : qualité et tests ;
- `.github/workflows/release.yml` : publication PyPI et GitHub Release ;
- `.github/CODEOWNERS` : propriétaires des revues ;
- `.github/dependabot.yml` : mises à jour automatisées ;
- `.github/PULL_REQUEST_TEMPLATE.md` : checklist imposée à chaque PR ;
- `.github/ISSUE_TEMPLATE/` : formulaires d'issues.

7. Publier une version

Prérequis unique : PyPI Trusted Publishing

Le workflow de release utilise OIDC : aucun token PyPI n'est stocké dans GitHub. Il faut déclarer le publisher une seule fois sur PyPI :

1. Se connecter sur `pypi.org` puis ouvrir *Account settings* → *Publishing*.
2. Créer un *pending publisher* avec : propriétaire `hamzasinko`, dépôt `dalil`, workflow `release.yml`, environnement `pypi`.
3. Créer l'environnement `pypi` dans *Settings* → *Environments* du dépôt (protection par approbation recommandée).

> Tant que ce publisher n'existe pas, tout déclenchement de `release.yml` échoue au job `Publish`. C'est la raison pour laquelle `prod` n'a pas encore été avancée.

Procédure de release

1. Mettre à jour `__version__` dans `src/dalil/__init__.py` et `CHANGELOG.md`.
2. Merger `dev` → `main` via PR.
3. Merger `main` → `prod` via PR : le push déclenche `release.yml`.
4. Le workflow teste, construit, publie sur PyPI, puis crée la release GitHub avec les notes générées.

Alternative : pousser un tag `v0.2.0` déclenche le même workflow.

Ordre des opérations

```
feature/* --PR--> dev --PR--> main --PR--> prod --CI/CD--> PyPI + GitHub Release
```

8. Roadmap suggérée

Chaque étape correspond à une branche `feature/*` créée depuis `dev`, avec tests nominaux et cas limites, documentation et entrée dans `CHANGELOG.md`.

Version	Branche	Livrable	Dépend de
0.2.0	`feature/flowchart-core`	AST → `FlowGraph` : entrée/sortie, conditions, boucles, appels	—
0.2.0	`feature/flowchart-renderers`	Exports DOT, Mermaid, puis SVG (extra `viz`)	flowchart-core
0.3.0	`feature/tracer-core`	Trace ligne par ligne, snapshots de locales, restauration garantie de `sys.settrace`	—
0.3.0	`feature/async-generators`	Support `async`, générateurs et appels concurrents	tracer-core
0.4.0	`feature/complexity-engine`	Estimation temps/espace : boucles, imbrication, récursion	—
0.4.0	`feature/latex-export`	Génération `algorithm2e` et `algorithmicx`	flowchart-core
0.5.0	`feature/html-report`	Rapport HTML autonome assemblant trace, graphe et complexité	tracer + flowchart + complexity
0.5.0	`feature/cli`	Commande `dalil` avec sortie `rich` (extra `cli`)	couches métier
0.6.0	`feature/docs-site`	`mkdocs.yml`, navigation, GitHub Pages	—
0.6.0	`feature/security-hardening`	Revue des imports dynamiques, timeouts, limites de taille de rapport	toutes
1.0.0	`feature/release-1-0`	Stabilité de l'API, benchmarks, documentation complète, première release PyPI	toutes

Critères d'acceptation par phase

0.2.0 - Flowchart

- un nœud d'entrée et de sortie unique pour toute fonction ;
- les branches `if/elif/else`, boucles `for/while` et `return` produisent des arêtes correctes ;
- les sorties DOT et Mermaid sont déterministes (tri stable) ;
- cas limites : fonction vide, `pass`, `try/except`, lambdas ;
- tests de propriété Hypothesis sur le nombre d'arêtes et la connexité.

0.3.0 - Tracer

- aucune modification du comportement, de la valeur de retour ni des exceptions ;
- `sys.settrace` restauré même en cas d'exception ou de thread ;

- historique limité en mémoire (option de taille maximale) ;
- cas limites : récursion, générateurs, async, décorateurs empilés.

0.4.0 - Complexité et LaTeX

- notation O(...) justifiée par la structure détectée ;
- sortie LaTeX compilable avec `algorithm2e` et `algorithmicx` ;
- échappement correct des identifiants et des chaînes.

0.5.0 - Rapport et CLI

- HTML autonome (CSS inline, aucune ressource réseau) ;
- échappement systématique des valeurs utilisateur (XSS) ;
- CLI testée avec `capsys` et codes de sortie stables.

1.0.0

- couverture ≥ 90 % maintenue, documentation publiée, API gelée ;
- publication PyPI via Trusted Publishing et notes de release automatiques.

9. Checklist et dépannage

Checklist avant chaque pull request

- branche créée depuis `dev` et PR ciblant `dev` ;
- titre de PR au format Conventional Commits ;
- tests nominaux et cas limites ajoutés ;
- `python -m pytest` vert avec couverture ≥ 90 % ;
- `ruff check`, `ruff format --check` et `mypy --strict src tests` verts ;
- documentation et `CHANGELOG.md` mis à jour ;
- template de PR rempli, issue liée (`Closes #123`) ;
- CI verte sur les 15 combinaisons + lint + build.

Erreurs fréquentes

Symptôme	Cause probable	Solution
`ModuleNotFoundError: dalil`	Package non installé	`python -m pip install -e ".[dev]"`
Couverture sous 90 %	Nouveau code non testé	Ajouter tests unitaires + edge
`mypy` échoue sur un test	Typage incomplet	Annoter les fixtures et helpers
ruff D100/D103	Docstring manquante	Ajouter une docstring Google (tests : D100/D103 ignorés)
PR bloquée "review required"	Compte solo	Utiliser le bypass admin ou ajouter un reviewer
Release échoue à `Publish`	Publisher PyPI absent	Configurer Trusted Publishing (section 7)
`posting review` impossible	Auto-approbation interdite	Faire relire la PR
Push refusé sur `main` / `dev` / `prod`	Branche protégée	Passer par une PR
PowerShell bloque `Activate.ps1`	ExecutionPolicy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`git` : dubious ownership	Dépôt appartenant à un autre utilisateur	`git config --global --add safe.directory <chemin>`

Liens utiles

Ressource	Adresse
Dépôt	https://github.com/hamzasinko/dalil
Issues	https://github.com/hamzasinko/dalil/issues
Discussions	https://github.com/hamzasinko/dalil/discussions

Actions	https://github.com/hamzasinko/dalil/actions
Signaler une vulnérabilité	https://github.com/hamzasinko/dalil/security/advisories/new
PyPI	https://pypi.org/project/dalil/ (après première release)
Conventional Commits	https://www.conventionalcommits.org/
Keep a Changelog	https://keepachangelog.com/
